

ENCAPSULATION:

```
class Student {
    private String name;
    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.setName("Janani");
        System.out.println("Name: " + s.getName());
    }
}
```

OUTPUT:

Name: Janani

ABSTRACTION:

```
abstract class Animal {
    abstract void sound();
    void sleep() {
        System.out.println("Animal is sleeping");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
```

```
Dog d = new Dog();  
d.sound();  
d.sleep();  
}  
}
```

OUTPUT:

Dog barks
Animal is sleeping

SINGLE INHERITANCE:

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

OUTPUT:

Animal is eating
Dog is barking

MULTILEVEL INHERITANCE:

```
class Animal {  
    void eat() {
```

```

        System.out.println("Animal is eating");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy is weeping");
    }
}
public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.weep();
    }
}

```

OUTPUT:

Animal is eating
 Dog is barking
 Puppy is weeping

HIERARCHICAL INHERITANCE:

```

class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}
class Dog extends Animal {
    void bark() {

```

```

        System.out.println("Dog is barking");
    }
}
class Cat extends Animal {
    void meow() {
        System.out.println("Cat is meowing");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();
        d.eat();
        d.bark();
        c.eat();
        c.meow();
    }
}

```

OUTPUT:

```

Animal is eating
Dog is barking
Animal is eating
Cat is meowing

```

SUPER KEYWORD:

```

class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

```

```
        void display() {  
            super.sound();  
        }  
    }  
  
    public class Main {  
        public static void main(String[] args) {  
            Dog d = new Dog();  
            d.display();  
        }  
    }
```

OUTPUT:

Animal makes sound

DEFAULT CONSTRUCTOR:

```
class Student {  
    String name;  
    Student() {  
        name ="Janani";  
    }  
    void display() {  
        System.out.println(name);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.display();  
    }  
}
```

OUTPUT:

Janani

PARAMETERIZED CONSTRUCTOR:

```
class Student {
```

```

String name;
Student(String name) {
    this.name = name;
}
void display() {
    System.out.println(name);
}
}
public class Main {
    public static void main(String[] args) {
        Student s = new Student("Janani");
        s.display();
    }
}

```

OUTPUT:

Janani

METHOD OVERLOADING:

```

class Calculator {
    void add(int a, int b) {
        System.out.println(a + b);
    }
    void add(int a, int b, int c) {
        System.out.println(a + b + c);
    }
    void add(double a, double b) {
        System.out.println(a + b);
    }
}
public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        c.add(4, 2);
        c.add(1, 2, 3);
    }
}

```

```
        c.add(4.2, 2.4);
    }
}
```

OUTPUT:

```
6
6
6.6
```

METHOD OVERRIDING:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

OUTPUT:

```
Dog barks
```